



SE618 LOGISTIC CASE STUDY ANALYSIS

WEEK 9 – PROGRESS REPORT

PROJECT TITLE: SERGE – AI Agent for Supply Chain Operations

Automated Evaluation Pipeline & LLM-as-Judge Implementation

Gauthier SCHMITT	6822807466
Louis GRIGNOLA	6822807516
Matteo SCHWARZ	6822807557
Maël TELLUS	6822807532

SUBMITTED TO:

AJARN WARUT

APRIL 2026

SEMESTER 2/2025

SCHOOL OF MANUFACTURING SYSTEMS AND MECHANICAL ENGINEERING
SIRINDHORN INTERNATIONAL INSTITUTE OF TECHNOLOGY
THAMMASAT UNIVERSITY

Contents

1	Executive Summary	2
2	Evaluation Pipeline Design	2
2.1	Motivation	2
2.2	Architecture Overview	2
2.3	Why Claude Code CLI, Not a Custom Agent	3
3	LLM-as-Judge Implementation	4
3.1	Judge Design	4
3.2	Structured Output Enforcement	4
3.3	Scenario Format	5
4	Pipeline Validation	5
4.1	End-to-End Test	5
4.2	Observations from Validation	6
5	Current Scenario Coverage	6
6	Metrics Framework	7
7	File Structure	7
8	Next Steps	8

Executive Summary

This week focused on building the **automated evaluation pipeline** for SERGE. The pipeline orchestrates end-to-end evaluation: it runs the agent on each scenario, captures structured outputs, sends them to an LLM-as-Judge for scoring, and aggregates results into a final report — all from a single shell command. The pipeline has been validated end-to-end on a test scenario. The current 15 evaluation scenarios serve as a working draft; next week, they will be reviewed, refined with realistic business questions, and used for the full 45-run evaluation.

Evaluation Pipeline Design

Motivation

Last week’s qualitative testing showed that SERGE can perform cross-system reasoning, but results were evaluated manually and lacked reproducibility. To produce rigorous, publishable results for the final report, we need:

1. **Automation:** run all scenarios without manual intervention.
2. **Structured scoring:** consistent, repeatable evaluation using a fixed rubric.
3. **Consistency measurement:** run each scenario multiple times to measure variance.
4. **Cost tracking:** monitor API costs across the full evaluation.

Architecture Overview

The pipeline runs entirely via the Claude Code CLI (`claude -p`) — no custom Go agent code is needed beyond the existing MCP server. The architecture has three sequential stages:

For each scenario (15) x each run (3) = 45 runs:

STEP 1: AGENT RUN

```

claude -p "<question>"
  --mcp-config .mcp.json           (supply chain MCP server)
  --allowedTools "mcp__serge-*"    (read-only SC tools only)
  --output-format json             (structured capture)
-> {agent_answer, duration_ms, cost_usd, num_turns}

```

STEP 2: JUDGE RUN

```

claude -p "<agent_answer + ground_truth + rubric>"
  --json-schema judge_schema.json (enforced JSON output)
  --tools ""                       (no tools, pure evaluation)
-> {factual_accuracy, completeness, reasoning, ...}

```

STEP 3: AGGREGATION (Python, deterministic)

```

-> evaluation_report.json

```

Figure 1: Three-stage evaluation pipeline. Steps 1 and 2 use the Claude Code CLI; Step 3 is a deterministic Python aggregation script.

Key Insight

The aggregation step was initially designed as an LLM call, but we replaced it with a deterministic Python script. Aggregation is pure arithmetic (averages, standard deviations, pass rates) — using an LLM would add cost and non-determinism for no benefit. The LLM is reserved for tasks that require judgement (Steps 1 and 2).

Why Claude Code CLI, Not a Custom Agent

We evaluated two approaches for running the evaluation:

Table 1: Comparison of evaluation approaches.

Criterion	Custom Go Agent (trpc-agent-go)	Claude Code CLI
MCP integration	Must implement MCP client	Built-in via <code>-mcp-config</code>
Tool permissions	Must implement permission system	Built-in via <code>-allowedTools</code>
Structured output	Must parse LLM responses	Built-in via <code>-json-schema</code>
Cost tracking	Must implement token counting	Built-in in JSON output
Development time	Days to weeks	Hours

Claude Code CLI provides all the infrastructure we need out of the box: MCP server management, tool permissions, structured output validation, cost tracking, and session isolation. Building a custom agent would duplicate this functionality with significant development overhead and no added capability for our evaluation use case.

LLM-as-Judge Implementation

Judge Design

The LLM-as-Judge evaluates each agent answer against a known ground truth using five weighted criteria:

Table 2: Evaluation rubric: five criteria with weights.

Criterion	Weight	What it measures
Factual Accuracy	30%	Are all stated facts correct? No errors or contradictions.
Completeness	25%	Are all ground truth facts present in the answer?
Reasoning Quality	20%	Is the causal chain sound? Logical connections between data?
Source Attribution	15%	Does the agent cite which system (SRM/WMS/OMS/TMS) data comes from?
Response Structure	10%	Is the response well-organized, scannable, with tables/lists?

Each criterion is scored 1–5. The weighted score is computed as:

$$\text{score} = 0.30 \times \text{accuracy} + 0.25 \times \text{completeness} + 0.20 \times \text{reasoning} + 0.15 \times \text{attribution} + 0.10 \times \text{structure}$$

A scenario **passes** if the weighted score ≥ 3.5 out of 5.0.

Structured Output Enforcement

The judge is invoked with `-json-schema`, which forces the LLM to produce a valid JSON object matching our schema:

```
{
  "factual_accuracy": 4,
  "completeness": 2,
  "reasoning": 4,
  "source_attribution": 2,
  "structure": 4,
  "weighted_score": 3.20,
  "pass": false,
  "justification": "All stated facts correct but missing...",
  "facts_found": ["status is delivered", "dates correct"],
  "facts_missing": ["total amount", "carrier name"],
  "hallucinations": []
}
```

This structured output enables automated aggregation without any manual parsing.

Scenario Format

Each evaluation scenario is stored as a JSON file containing the question, expected tool calls, and ground truth:

```
{
  "id": "uc2-01",
  "use_case": "UC-2: Root Cause Analysis",
  "title": "Stock Shortage - Order Blocked",
  "difficulty": "hard",
  "question": "Why hasn't order ORD-007 shipped yet?",
  "expected_tools": ["oms_get_order", "tms_track_order",
                    "wms_check_inventory", "srm_list..."],
  "ground_truth": {
    "facts": [
      "ORD-007 status is processing, priority urgent",
      "Needs 50 units of PROD-001, only 8 available",
      "PO-008 incoming with 150 units, ETA 2026-02-18",
      "Root cause: inventory shortage"
    ],
    "must_not_contain": [
      "Claim order can be fulfilled from current stock"
    ]
  }
}
```

Pipeline Validation

End-to-End Test

We validated the pipeline on scenario uc1-01 (Happy Path — Delivered Order) with 1 run:

Table 3: E2E validation results on uc1-01.

Metric	Value
Agent response time	12.1 seconds
Agent cost	\$0.058
Agent tool calls (turns)	3
Judge: Factual Accuracy	4/5 (all stated facts correct)
Judge: Completeness	2/5 (missing carrier, prices, total)
Judge: Reasoning	4/5
Judge: Source Attribution	2/5 (only OMS cited, TMS not queried)
Judge: Structure	4/5
Weighted Score	3.20/5.0
Pass	No (< 3.5 threshold)

Result

The pipeline successfully executed all three stages: agent run, judge evaluation, and aggregation. The judge correctly identified that the agent did not call `tms_track_order` (only OMS was queried), resulting in missing shipment details (SHP-001, carrier SCG Cold Chain). The structured JSON output was produced and aggregated without errors.

Observations from Validation

1. **Agent tool selection varies across runs.** In some runs the agent calls both `oms_get_order` and `tms_track_order`; in others it stops after OMS. This is the consistency issue we will quantify with 3 runs per scenario.
2. **The judge is strict and fair.** It gave a high accuracy score (4/5) because every stated fact was correct, but penalised completeness (2/5) for missing ground truth facts. This aligns with our rubric design.
3. **Cost is manageable.** At \$0.06 per agent run + \$0.06 per judge run, the full evaluation (45 + 45 = 90 calls) should cost approximately \$5–8 total.
4. **Authentication matters.** The `-bare` flag (which skips all local configuration) also skips OAuth authentication. We switched to individual flags (`-strict-mcp-config`, `-disable-slash-commands`) to maintain authentication while still isolating the evaluation environment.

Current Scenario Coverage

The 15 draft scenarios cover three use cases across three difficulty levels:

Table 4: Current 15 evaluation scenarios (draft — to be refined next week).

ID	Use Case	Title	Difficulty	Systems
uc1-01	Order Status	Happy path (ORD-001)	Easy	OMS, TMS
uc1-02	Order Status	In transit (ORD-005)	Easy	OMS, TMS
uc1-03	Order Status	Exception (ORD-011)	Medium	OMS, TMS
uc1-04	Order Status	Cancelled (ORD-014)	Easy	OMS, TMS
uc1-05	Order Status	Late delivery (ORD-003)	Medium	OMS, TMS
uc2-01	Root Cause	Stock shortage (ORD-007)	Hard	4 systems
uc2-02	Root Cause	Supplier delay pattern	Medium	SRM
uc2-03	Root Cause	Carrier exception (SHP-009)	Medium	TMS, OMS
uc2-04	Root Cause	Order stuck (ORD-012)	Medium	OMS, WMS, TMS
uc2-05	Root Cause	Operational overview	Hard	4 systems
uc3-01	Supplier Comp.	Compare all suppliers	Easy	SRM
uc3-02	Supplier Comp.	Best reliability	Easy	SRM
uc3-03	Supplier Comp.	Spending by supplier	Medium	SRM
uc3-04	Supplier Comp.	Fastest lead time	Easy	SRM
uc3-05	Supplier Comp.	Late delivery history	Medium	SRM

Draft Status

These scenarios were designed alongside the seed data and serve as a functional proof-of-concept. Next week, we will review and refine them to ensure they reflect realistic business questions that a supply chain operator would actually ask, with accurate and complete ground truth derived from the seed data.

Metrics Framework

The full evaluation will measure six metrics:

Table 5: Six evaluation metrics.		
Metric	Source	Description
Judge Score	Judge output	Weighted average of 5 criteria (1–5 scale)
Factual Accuracy	Judge: facts_found	% of ground truth facts correctly stated
Pass Rate	Judge: pass	% of runs scoring ≥ 3.5
Tool Efficiency	Agent: num_turns	Number of tool calls vs. expected
Response Time	Agent: duration_ms	Median response time by difficulty
Consistency	Std. deviation	σ of weighted scores across 3 runs per scenario

The aggregation script computes these automatically, grouped by scenario, use case, and difficulty level.

File Structure

```
serge/eval/  
  config.json           Config (runs, models, budgets, weights)  
  judge_prompt.md       LLM-as-Judge system prompt  
  judge_schema.json     JSON Schema for structured judge output  
  aggregate.py          Deterministic Python aggregation  
  run.sh                Orchestrator (bash)  
  scenarios/            15 JSON scenario files  
    uc1-01-happy-path.json ... uc3-05-late-delivery-history.json  
  results/              Generated per run (gitignored)  
    <scenario_id>/run_N.json + run_N_judge.json  
  report/  
    evaluation_report.json Aggregated report (gitignored)
```

Pipeline commands:

```
./eval/run.sh                # Full: 15x3 = 45 runs  
./eval/run.sh --scenario uc1-01-happy-path # Single scenario  
./eval/run.sh --judge-only    # Re-judge existing results  
./eval/run.sh --aggregate-only # Re-aggregate only  
./eval/run.sh --dry-run       # Preview without API calls
```

Next Steps

Table 6: Remaining work (1 week).

Week	Dates	Deliverables
10	Apr 7 – 16	1. Scenario refinement: Review all 15 scenarios with the team. Ensure questions reflect realistic operator needs. Verify ground truth against seed data. Add edge cases if needed. 2. Full evaluation run: Execute 45 runs (15 scenarios × 3 runs). 3. Results analysis: Analyse pass rates, consistency, failure modes. Produce visualisations (score distributions, per-UC breakdown). 4. Final report & presentation: Write final report with methodology, results, and conclusions. Prepare demo.

Priority for Next Week

The most critical task is **scenario refinement**. The pipeline is ready — the quality of the evaluation now depends entirely on the quality of the questions and ground truth. Each scenario must be reviewed for: (1) realistic business framing, (2) accurate ground truth numbers verified against the seed JSON, (3) appropriate difficulty calibration, and (4) clear expected tool call sequences.